**PROGRAMOWANIE V**

# Projekt 1. Uniwersalny konwerter plików

*Laboratoria 2–5 | praca z formatami, bibliotekami zewnętrznymi i GUI*

| | |
|---|---|
| **Forma** | projekt aplikacyjny |
| **Technologia bazowa** | Python 3 + Tkinter/ttk |
| **Punktacja** | 100 pkt |
| **Wersja dokumentu** | 5 października 2026 |

*Dokument dla studentów i arkusz oceny dla prowadzącej.*

# Cel i scenariusz

Zaprojektuj aplikację desktopową, która pozwala wybrać plik wejściowy, pokazuje wyłącznie dostępne dla niego formaty docelowe, wykonuje konwersję i zapisuje wynik we wskazanym katalogu. Projekt ma utrwalić podstawy Pythona, pracę z systemem plików, integrację bibliotek i narzędzi zewnętrznych oraz obsługę błędów w aplikacji z GUI.

**Rekomendowany zakres:** aplikacja powinna wspierać co najmniej 6 kierunków konwersji z trzech grup: dane strukturalne, tekst/dokument oraz dokument/PDF. Zakres musi być jednoznacznie opisany w README.

# Wymagania funkcjonalne

- **Grupa A – dane strukturalne:** obowiązkowo CSV → JSON i JSON → CSV; należy zdefiniować i opisać akceptowaną strukturę JSON, kodowanie i sposób obsługi nagłówków.
- **Grupa B – tekst i dokumenty:** co najmniej dwa kierunki, np. TXT → DOCX, TXT → ODT, TXT → PDF, DOCX → TXT albo ODT → TXT.
- **Grupa C – formaty biurowe:** co najmniej dwa kierunki, np. DOCX → PDF, ODT → PDF, DOCX → ODT, ODT → DOCX; dopuszczalne jest użycie LibreOffice w trybie bez interfejsu.
- **Interfejs:** wybór pliku, katalogu zapisu i formatu docelowego; przycisk uruchomienia; czytelny status oraz komunikaty powodzenia i błędu.
- **Walidacja:** sprawdzenie istnienia pliku, rozszerzenia, dostępności konwertera, możliwości zapisu, konfliktu nazw i niepustych wyborów.
- **Jakość wyniku:** poprawne polskie znaki; zachowanie kolejności i nazw kolumn dla CSV/JSON; sensowne przeniesienie akapitów dla dokumentów tekstowych.
- **Architektura:** logika konwersji ma być oddzielona od kodu GUI; dodanie nowej pary formatów nie powinno wymagać przebudowania całego interfejsu.

## Zakres poza obowiązkiem

Nie wymaga się zachowania złożonego formatowania, osadzonych wykresów, przypisów, makr ani identycznego układu stron po konwersji. Nie należy obiecywać „dowolnej konwersji”; aplikacja ma jasno prezentować macierz wspieranych par.

# Harmonogram i kamienie milowe

| Zajęcia | Zakres pracy | Rezultat do pokazania | Kryterium zaliczenia etapu |
|---|---|---|---|
| Lab. 2 | TXT, CSV i JSON; projekt macierzy konwersji; repozytorium. | Działające CSV → JSON i JSON → CSV uruchamiane bez GUI; przykładowe dane. | Wyniki można ponownie odczytać; zachowane są nagłówki i znaki UTF-8; błędny JSON daje kontrolowany komunikat. |

| Zajęcia | Zakres pracy | Rezultat do pokazania | Kryterium zaliczenia etapu |
|---|---|---|---|
| Lab. 3 | DOCX, ODT i PDF; wybór bibliotek/narzędzia; kolejne konwertery. | Minimum cztery działające kierunki łącznie; demonstracja wywołania narzędzia zewnętrznego. | Brak użycia shell=True; sprawdzany jest kod zakończenia; brak narzędzia jest wykrywany i opisany. |
| Lab. 4 | GUI w Tkinter/ttk; integracja rejestru konwerterów. | Wybór pliku i katalogu; formaty docelowe zależne od rozszerzenia; wykonanie min. jednej konwersji. | GUI nie przyjmuje niedozwolonej pary; nie nadpisuje pliku bez pytania; komunikaty są zrozumiałe. |
| Lab. 5 | Walidacja, wyjątki, testy, dokumentacja i oddanie. | Kompletna aplikacja, 6 kierunków, README, zależności i dane testowe. | Uruchomienie według README na czystym środowisku; przejście minimalnej macierzy testów i obrona. |

*Kamienie milowe służą do bieżącej informacji zwrotnej. Punkty za systematyczność są przyznawane w rubryce końcowej.*

## Minimalna macierz testów

| Przypadek | Oczekiwany rezultat | Dowód |
|---|---|---|
| Poprawny plik w każdej wspieranej grupie | Powstaje otwieralny plik o właściwej treści. | Plik wejściowy, wyjściowy i krótki opis. |
| Błędny lub uszkodzony plik | Brak awarii aplikacji; jednoznaczny komunikat. | Zrzut/raport testu. |
| Nieobsługiwana para | Opcja niewidoczna lub zablokowana; brak próby konwersji. | Demonstracja w GUI. |
| Brak LibreOffice/biblioteki | Kontrolowany błąd z instrukcją rozwiązania. | Test przez zmianę konfiguracji. |
| Plik docelowy już istnieje | Pytanie o nadpisanie albo automatycznie bezpieczna nazwa. | Demonstracja. |
| Polskie znaki i puste wartości | Znaki i struktura danych pozostają poprawne. | Porównanie wejścia i wyjścia. |

## Kryteria oceny

Projekt jest oceniany w skali 0–100 pkt. Punkty przyznaje się na podstawie działającej wersji, kodu źródłowego, dokumentacji i krótkiej obrony.

| Obszar | Maks. | Pełna punktacja | Zasady punktacji częściowej |
|---|---|---|---|
| Konwersje i poprawność wyników | 30 | 6 kierunków z 3 grup działa poprawnie dla danych typowych i brzegowych; wyniki są otwieralne, zgodne z udokumentowaną semantyką. | 5 pkt za każdą poprawną parę do 30 pkt; para z istotną utratą treści maks. 2 pkt; para tylko zadeklarowana 0 pkt. |
| Architektura i kod | 15 | Oddzielone GUI, walidacja i konwertery; rejestr/dispatcher; czytelne funkcje, pathlib, brak duplikacji i bezpieczne wywołanie procesu. | 10–14 pkt: drobne sprzężenia; 5–9 pkt: logika działa, ale jest monolityczna; 0–4 pkt: kod trudny do uruchomienia lub zrozumienia. |
| Walidacja i błędy | 15 | Obsłużono wszystkie pozycje z minimalnej macierzy; komunikaty zawierają przyczynę i dalszy krok; błędy są logowane. | Po 2–3 pkt za: wejście, para formatów, zapis/nadpisanie, błędna treść, brak zależności, kontrola procesu. |
| GUI i użyteczność | 10 | Pełny przepływ bez konsoli; dynamiczne formaty; stan pracy; spójny układ i blokada błędnych operacji. | 7–9 pkt: drobne problemy UX; 4–6 pkt: przepływ możliwy, ale nieczytelny; 0–3 pkt: GUI nie obsługuje głównego scenariusza. |
| Testy | 10 | Co najmniej 6 testów obejmujących poprawne i błędne dane; część logiki testowana automatycznie; wyniki opisane. | 1 pkt za udokumentowany przypadek do 6 pkt + 4 pkt za sensowne testy automatyczne. |

| Obszar | Maks. | Pełna punktacja | Zasady punktacji częściowej |
|---|---|---|---|
| Dokumentacja i odtwarzalność | 10 | README, requirements/pyproject, macierz formatów, zależności systemowe, instrukcja instalacji i przykłady; projekt uruchamia się według opisu. | Brak README: 0–2 pkt; brak zależności lub macierzy: odjęcie 2 pkt za każdy element; nieaktualna instrukcja: maks. 5 pkt. |
| Systematyczność i repozytorium | 5 | Kamienie milowe pokazane terminowo; logiczne commity/dziennik zmian; brak jednorazowego zrzutu całego projektu. | 1 pkt za każdy udokumentowany etap oraz 1 pkt za czytelną historię zmian. |
| Obrona i samodzielność | 5 | Student wyjaśnia przepływ konwersji, decyzje biblioteczne i obsługę wyjątku oraz wykonuje małą zmianę. | 3–4 pkt: częściowe wyjaśnienie; 1–2 pkt: znaczna pomoc; 0 pkt: brak potwierdzenia rozumienia. |
| **RAZEM** | **100** | | |

## Progi ocen i warunki

| Punkty | Ocena |
|---|---|
| 0–49 | 2,0 |
| 50–59 | 3,0 |
| 60–69 | 3,5 |
| 70–79 | 4,0 |
| 80–89 | 4,5 |
| 90–100 | 5,0 |

- **Warunek zaliczenia:** co najmniej 50 pkt oraz działająca, możliwa do uruchomienia wersja realizująca funkcję główną projektu.
- **Brak wersji uruchamialnej:** maksymalna ocena 2,0, jeżeli projektu nie da się ocenić mimo zastosowania instrukcji z README.
- **Samodzielność:** student musi wyjaśnić wskazany fragment kodu i wprowadzić niewielką zmianę podczas obrony; brak potwierdzenia autorstwa oznacza wstrzymanie oceny do ponownej weryfikacji.
- **Praca w parze:** wspólna może być ocena produktu, ale punkty za obronę i wkład są indywidualne; wymagany jest opis podziału pracy.

- **Biblioteki i generatory AI:** wolno z nich korzystać, ale należy je ujawnić w README, podać źródła i rozumieć wykorzystany kod.

## Kod inspiracyjny

Rejestr konwerterów pozwala oddzielić logikę od GUI. Combobox może pobierać wartości z available_targets(), a przycisk „Konwertuj” wywoływać execute() w wątku roboczym.

> **Ważne:** Fragment pokazuje wzorzec rozwiązania, lecz nie stanowi kompletnej aplikacji. Miejsca TODO oraz integrację z GUI należy zaprojektować samodzielnie.

```
from dataclasses import dataclass
from pathlib import Path
from collections.abc import Callable
import subprocess

Converter = Callable[[Path, Path], Path]
REGISTRY: dict[tuple[str, str], Converter] = {}


def converter(source: str, target: str):
    """Dekorator rejestrujący pojedynczy kierunek konwersji."""
    def register(func: Converter) -> Converter:
        REGISTRY[(source.lower(), target.lower())] = func
        return func
    return register


@dataclass(frozen=True)
class ConversionRequest:
    source: Path
    target_format: str
    output_dir: Path

    def validate(self) -> None:
        if not self.source.is_file():
            raise FileNotFoundError("Nie znaleziono pliku wejściowego")
        if not self.output_dir.is_dir():
            raise NotADirectoryError("Nie znaleziono katalogu docelowego")
        key = (self.source.suffix.lower(), self.target_format.lower())
        if key not in REGISTRY:
            raise ValueError(f"Nieobsługiwana konwersja: {key}")


def available_targets(source: Path) -> list[str]:
    return sorted(target for (src, target) in REGISTRY if src == source.suffix.lower())


@converter(".docx", ".pdf")
def docx_to_pdf(source: Path, output_dir: Path) -> Path:
    command = [
        "libreoffice", "--headless", "--convert-to", "pdf",
        "--outdir", str(output_dir), str(source)
    ]
    result = subprocess.run(command, capture_output=True, text=True, timeout=60)
    if result.returncode != 0:
        raise RuntimeError(result.stderr.strip() or "Konwersja nie powiodła się")
    target = output_dir / f"{source.stem}.pdf"
    if not target.exists():
        raise RuntimeError("Narzędzie nie utworzyło oczekiwanego pliku")
    return target
```

```
def execute(request: ConversionRequest) -> Path:
    request.validate()
    key = (request.source.suffix.lower(), request.target_format.lower())
    # TODO: polityka nadpisywania i rejestrowanie czasu operacji
    return REGISTRY[key](request.source, request.output_dir)
```

### Co student ma doprojektować

- Zaimplementować konwertery danych strukturalnych i tekstowych oraz wspólny model błędów.
- Dodać strategię bezpiecznej nazwy albo potwierdzenie nadpisania.
- Wykryć ścieżkę do LibreOffice na różnych systemach i udokumentować zależność.
- Połączyć rejestr z Tkinter/ttk bez zamrażania okna podczas dłuższej operacji.
- Napisać testy dla validate(), available_targets() i co najmniej dwóch konwerterów.

## Pakiet do oddania

- Kod źródłowy w uporządkowanej strukturze katalogów, bez środowiska wirtualnego i dużych plików binarnych.
- README.md: cel, wymagania, instalacja, uruchomienie, obsługiwane funkcje, znane ograniczenia i autorzy.
- Plik requirements.txt lub pyproject.toml z zależnościami.
- Co najmniej trzy reprezentatywne dane testowe oraz opis oczekiwanego wyniku.
- Krótki raport testów i lista wykorzystanych źródeł, bibliotek oraz narzędzi AI.
- Historia pracy w repozytorium Git albo równoważny dziennik zmian.
- Tabela wspieranych par formatów wraz z informacją o znanych ograniczeniach każdej konwersji.

## Materiały techniczne

Poniższe oficjalne dokumentacje są punktem odniesienia; student może dobrać inne biblioteki, jeśli uzasadni wybór i zachowa wymagania projektu.

1. **Dokumentacja tkinter.filedialog** — natywne okna wyboru pliku, katalogu i ścieżki zapisu https://docs.python.org/3/library/dialog.html
2. **python-docx** — tworzenie, odczyt i modyfikowanie plików DOCX https://python-docx.readthedocs.io/
3. **LibreOffice – parametry wiersza poleceń** — uruchamianie pakietu w trybie headless i automatyzacja https://help.libreoffice.org/latest/en-US/text/shared/guide/start_parameters.html
4. **Moduł subprocess** — bezpieczne uruchamianie procesu zewnętrznego https://docs.python.org/3/library/subprocess.html

# Arkusz oceny

**Student/ka lub zespół:** _____

**Data obrony:** _____ **Prowadzący:** _____

| Kryterium | Maks. | Przyznano | Komentarz |
|---|---|---|---|
| Konwersje i poprawność wyników | 30 | | |
| Architektura i kod | 15 | | |
| Walidacja i błędy | 15 | | |
| GUI i użyteczność | 10 | | |
| Testy | 10 | | |
| Dokumentacja i odtwarzalność | 10 | | |
| Systematyczność i repozytorium | 5 | | |
| Obrona i samodzielność | 5 | | |
| RAZEM | 100 | | |

Ocena: _____ Podpis: _____

Uwagi do poprawy / uzasadnienie oceny:

_____

_____

_____

_____

_____